

CSE 385: Analysis of Algorithms (Honors)

Lecture 3 (Insertion Sort and Selection Sort)

Rezaul A. Chowdhury
Department of Computer Science
SUNY Stony Brook
Spring 2026

Insertion Sort

Input array

	1	2	3	4	5	6	7	8	9	10
A	7	4	1	15	9	5	12	3	18	11

Let **State j** be a state of the array A in which all numbers that were originally in $A[1..j]$ are placed in sorted order (i.e., nondecreasing order of value) in $A[1..j]$.

State 1

Input array

	1	2	3	4	5	6	7	8	9	10
A	7	4	1	15	9	5	12	3	18	11

sorted unchanged

State 5

Suppose somehow we have this:

	1	2	3	4	5	6	7	8	9	10
A	1	4	7	9	15	5	12	3	18	11

State 6

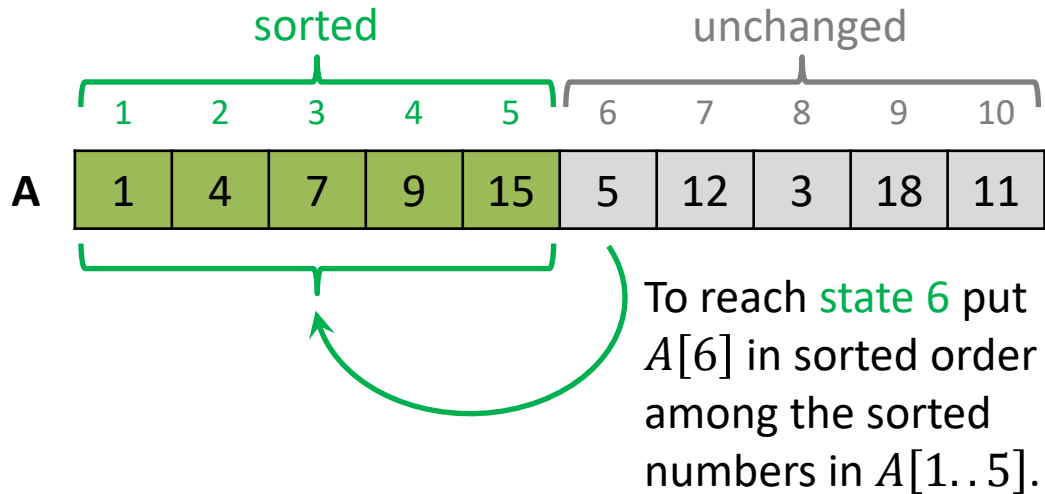
Now from state 5 we want to reach this:

	1	2	3	4	5	6	7	8	9	10
A	1	4	5	7	9	15	12	3	18	11

Insertion Sort

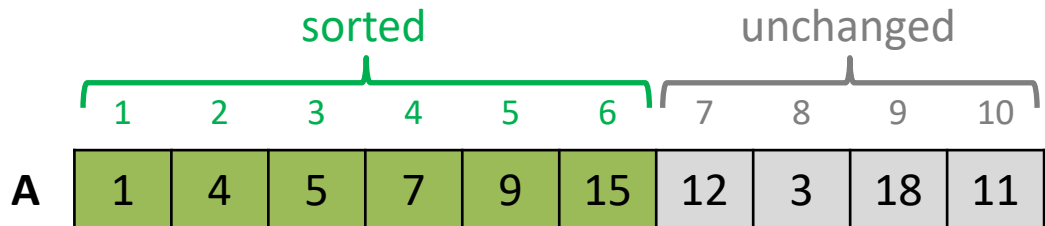
State 5

Suppose somehow we have this:



State 6

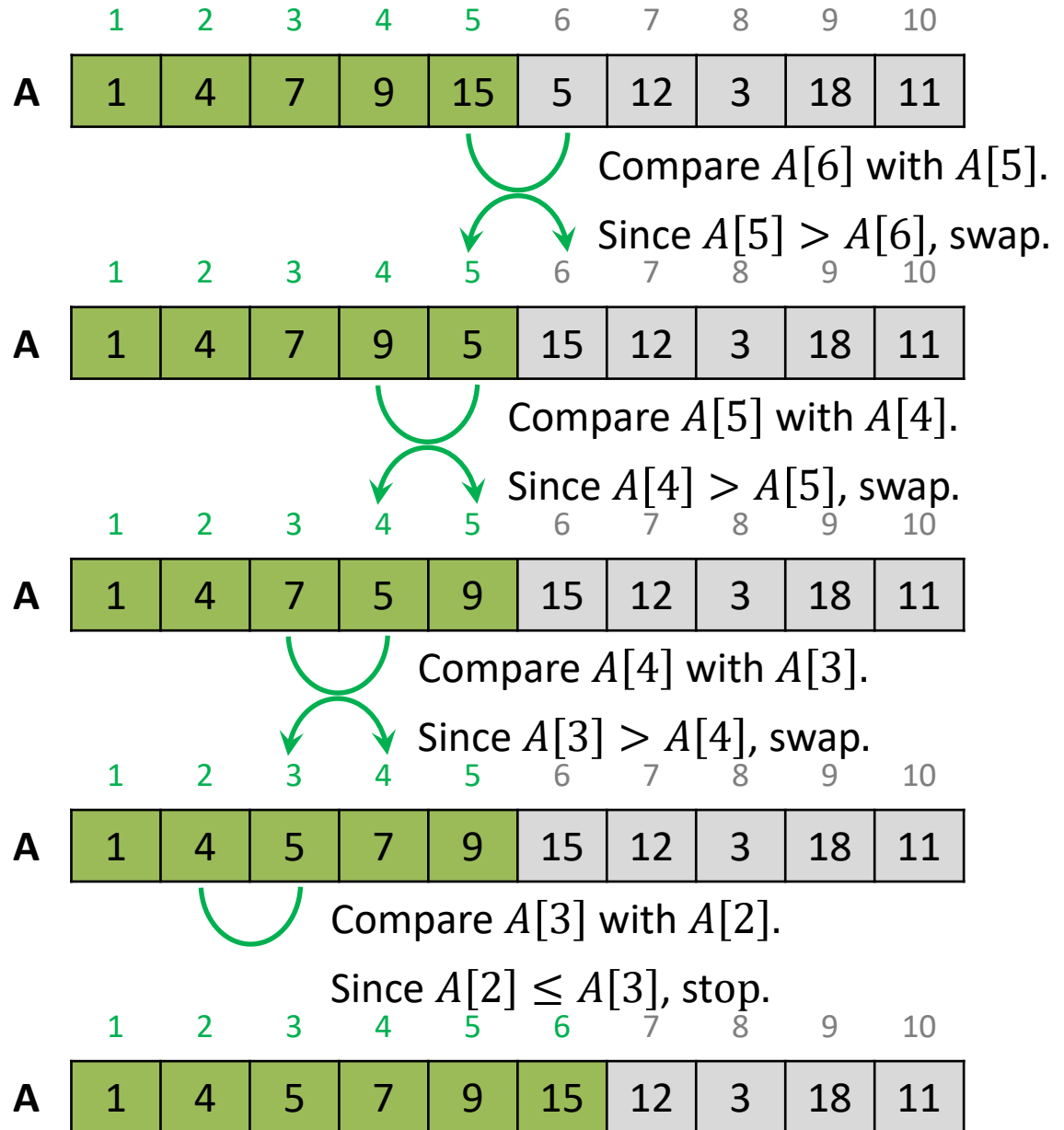
Now from state 5 we want reach this:



Insertion Sort

State 5

Suppose somehow we have this:



State 6

Insertion Sort

Input array

	1	2	3	4	5	6	7	8	9	10
A	7	4	1	15	9	5	12	3	18	11

State 1

$A[1]$ is trivially sorted

	1	2	3	4	5	6	7	8	9	10
A	7	4	1	15	9	5	12	3	18	11



Insert $A[2]$ in sorted order into $A[1]$

State 2

$A[1..2]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	4	7	1	15	9	5	12	3	18	11



Insert $A[3]$ in sorted order into $A[1..2]$

State 3

$A[1..3]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	4	7	15	9	5	12	3	18	11



Insert $A[4]$ in sorted order into $A[1..3]$

State 4

$A[1..4]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	4	7	15	9	5	12	3	18	11



Insert $A[5]$ in sorted order into $A[1..4]$

State 5

$A[1..5]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	4	7	9	15	5	12	3	18	11

Insertion Sort

State 5

$A[1..5]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	4	7	9	15	5	12	3	18	11



Insert $A[6]$ in sorted order into $A[1..5]$

State 6

$A[1..6]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	4	5	7	9	15	12	3	18	11



Insert $A[7]$ in sorted order into $A[1..6]$

State 7

$A[1..7]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	4	5	7	9	12	15	3	18	11



Insert $A[8]$ in sorted order into $A[1..7]$

State 8

$A[1..8]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	12	15	18	11



Insert $A[9]$ in sorted order into $A[1..8]$

State 9

$A[1..9]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	12	15	18	11



Insert $A[10]$ in sorted order into $A[1..9]$

State 10

$A[1..10]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	11	12	15	18

Insertion Sort

Input: An array $A[1 : n]$ of n numbers.

Output: Elements of $A[1 : n]$ rearranged in non-decreasing order of value.

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$
2. // insert $A[j]$ into the sorted sequence $A[1..j - 1]$
3. $i = j - 1$
4. **while** $i > 0$ **and** $A[i] > A[i + 1]$
5. $A[i + 1] \leftrightarrow A[i]$ // swap $A[i]$ and $A[i + 1]$
6. $i = i - 1$

Worst Case Runtime of Insertion Sort (Upper Bound)

INSERTION-SORT (A)

	<u>cost</u>	<u>times</u>
1. for $j = 2$ to $A.length$	c_1	n
2. $//$ insert $A[j]$ into the sorted sequence $A[1..j - 1]$	0	$n - 1$
3. $i = j - 1$	c_3	
4. while $i > 0$ and $A[i] > A[i + 1]$	c_4	$\sum_{2 \leq j \leq n} j$
5. $A[i + 1] \leftrightarrow A[i]$ $//$ swap $A[i]$ and $A[i + 1]$	c_5	$\sum_{2 \leq j \leq n} (j - 1)$
6. $i = i - 1$	c_6	

Running time, $T(n) \leq c_1 n + c_3(n - 1)$

$$+ c_4 \sum_{j=2}^n j + c_5 \sum_{j=2}^n (j - 1) + c_6 \sum_{j=2}^n (j - 1)$$

$$= 0.5(c_4 + c_5 + c_6)n^2 + 0.5(2c_1 + 2c_3 + c_4 - c_5 - c_6)n - (c_3 + c_4)$$

$$\Rightarrow T(n) = O(n^2)$$

Best Case Runtime of Insertion Sort (Lower Bound)

INSERTION-SORT (A)

	<u>cost</u>	<u>times</u>
1. for $j = 2$ to $A.length$	c_1	n
2. $//$ insert $A[j]$ into the sorted sequence $A[1..j - 1]$	0	$n - 1$
3. $i = j - 1$	c_3	
4. while $i > 0$ and $A[i] > A[i + 1]$	c_4	
5. $A[i + 1] \leftrightarrow A[i]$ $//$ swap $A[i]$ and $A[i + 1]$	c_5	0
6. $i = i - 1$	c_6	

Running time, $T(n) \geq c_1 n + c_3(n - 1) + c_4(n - 1)$

$$= (c_1 + c_3 + c_4)n - (c_3 + c_4)$$

$$\Rightarrow T(n) = \Omega(n)$$

Insertion Sort

(Slightly Optimized but Same Asymptotic Bounds)

Input: An array $A[1 : n]$ of n numbers.

Output: Elements of $A[1 : n]$ rearranged in non-decreasing order of value.

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$
2. $key = A[j]$
3. // insert $A[j]$ into the sorted sequence $A[1..j - 1]$
4. $i = j - 1$
5. **while** $i > 0$ **and** $A[i] > key$
6. $A[i + 1] = A[i]$
7. $i = i - 1$
8. $A[i + 1] = key$

Selection Sort

Input array

	1	2	3	4	5	6	7	8	9	10
A	7	4	1	15	9	5	12	3	18	11

Let **State j** be a state of the array A in which the smallest j numbers of $A[1..n]$ are placed in sorted order (i.e., nondecreasing order of value) in $A[1..j]$, and the remaining numbers placed in arbitrary order in $A[j + 1..n]$.

State 0

Input array

	1	2	3	4	5	6	7	8	9	10
A	7	4	1	15	9	5	12	3	18	11

smallest 4 numbers
in sorted order

remaining
6 numbers

State 4

Suppose somehow
we have this:

	1	2	3	4	5	6	7	8	9	10
A	1	3	4	5	9	15	12	7	18	11

smallest 5 numbers
in sorted order

remaining
5 numbers

State 5

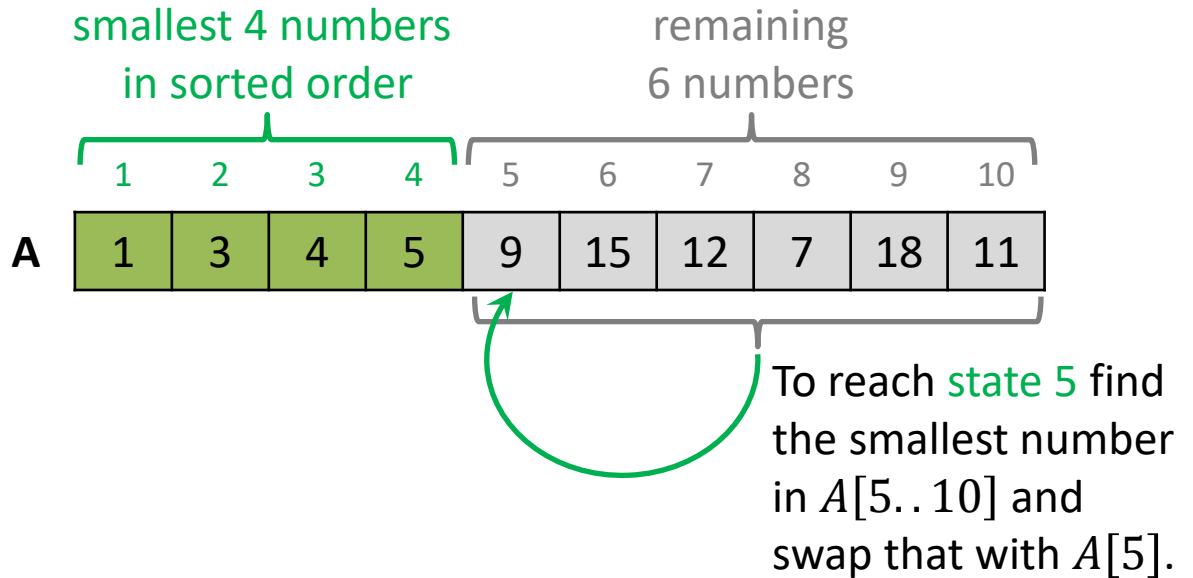
Now from state 4 we
want to reach this:

	1	2	3	4	5	6	7	8	9	10
A	1	3	4	5	7	15	12	9	18	11

Selection Sort

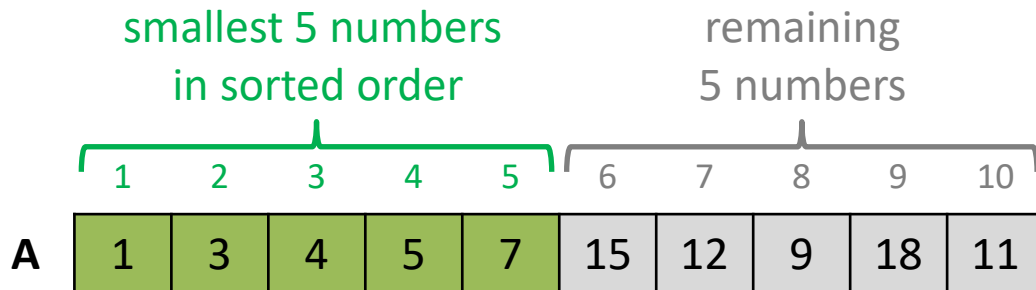
State 4

Suppose somehow we have this:



State 5

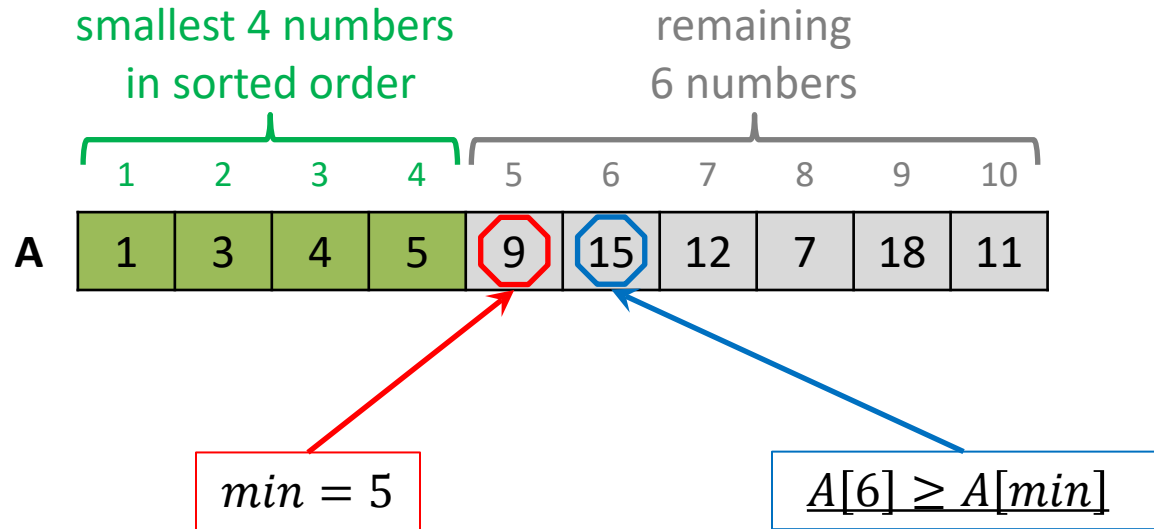
Now from state 4 we want reach this:



Selection Sort

State 4

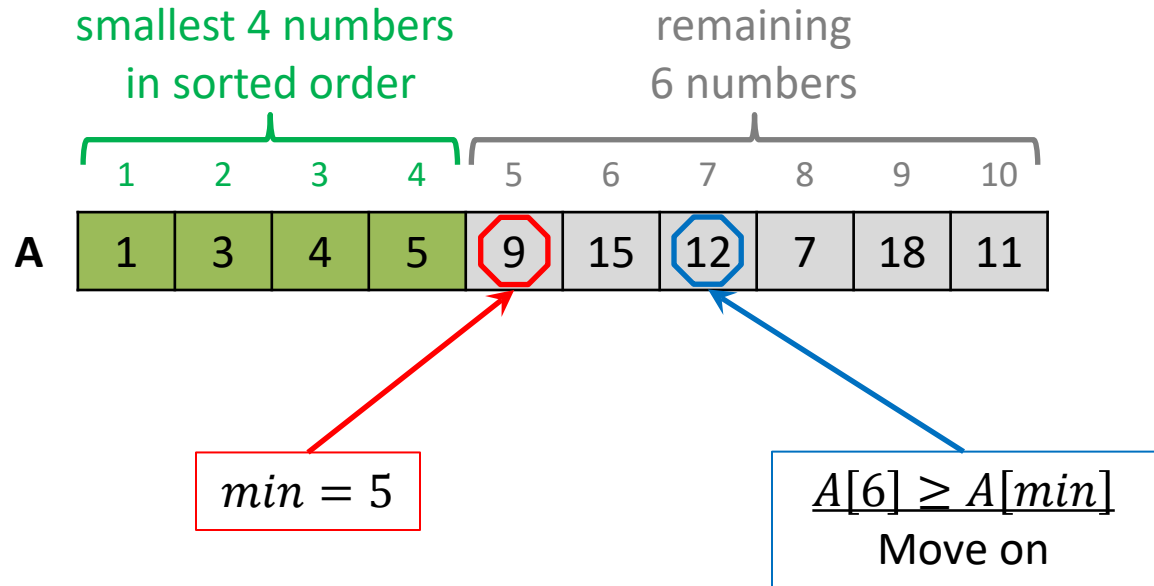
Suppose somehow we have this:



Selection Sort

State 4

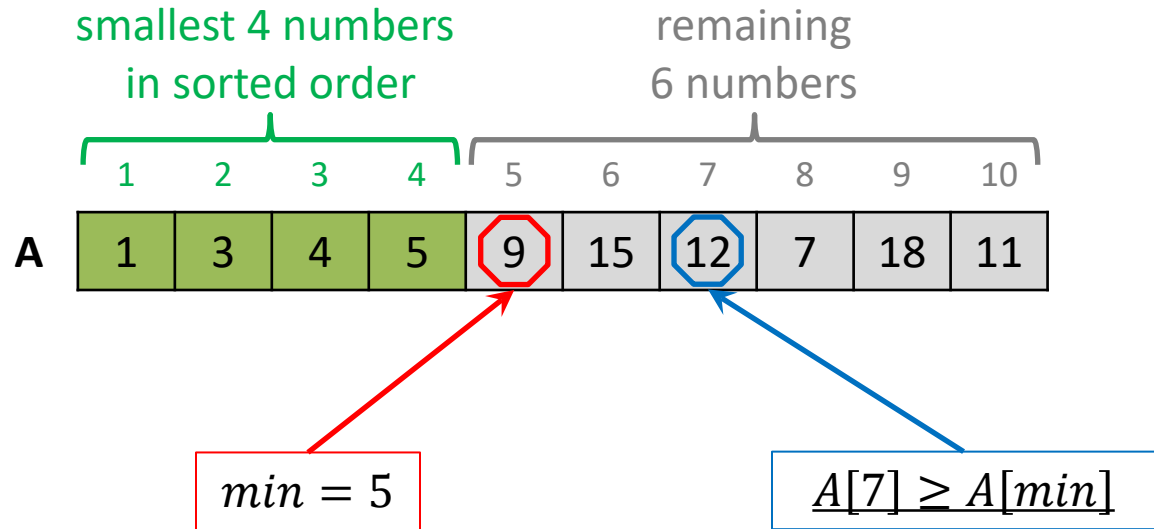
Suppose somehow we have this:



Selection Sort

State 4

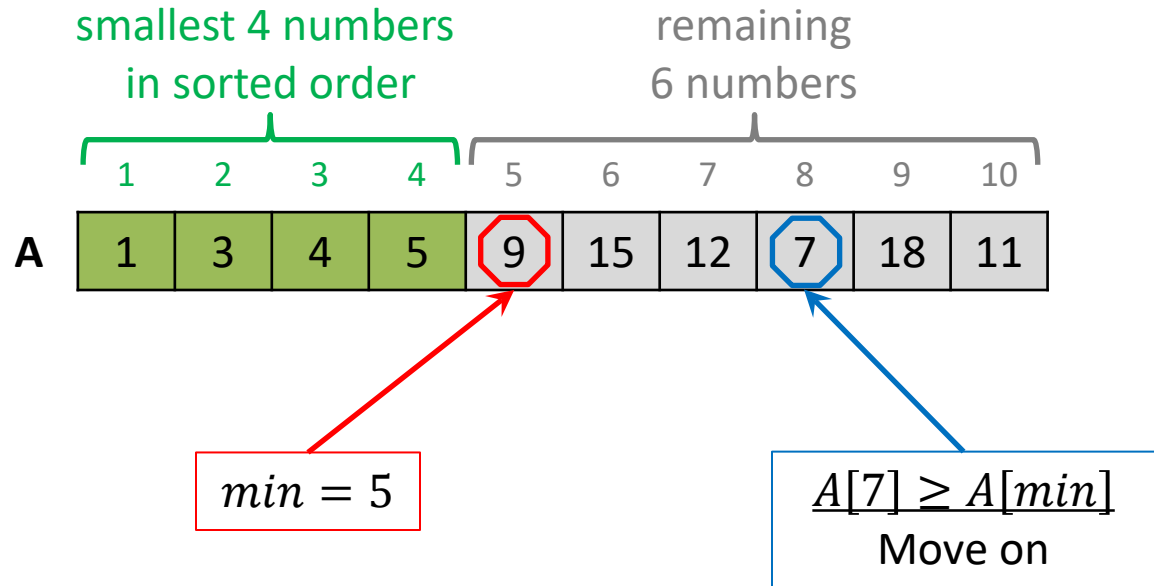
Suppose somehow we have this:



Selection Sort

State 4

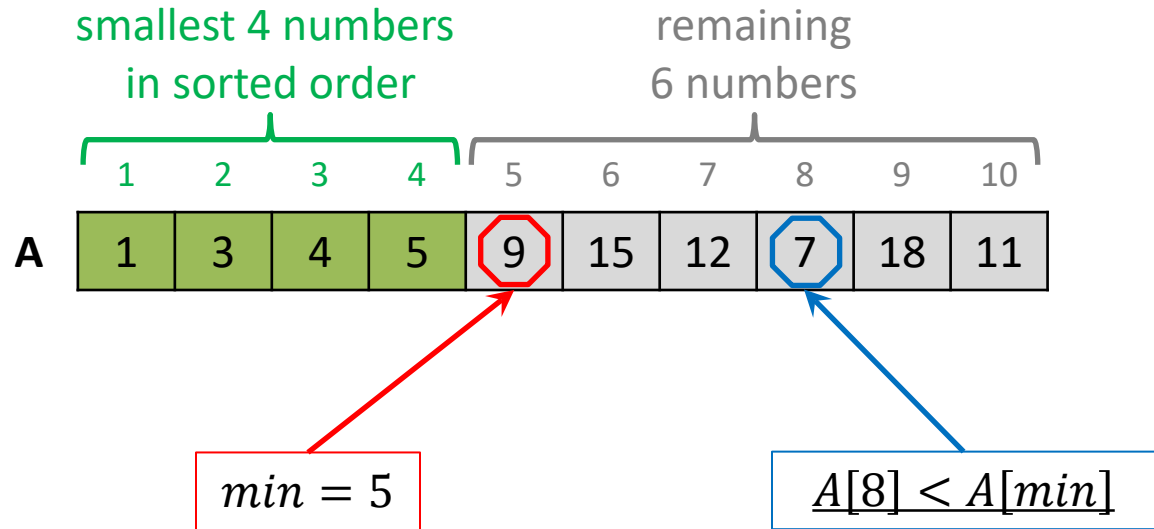
Suppose somehow we have this:



Selection Sort

State 4

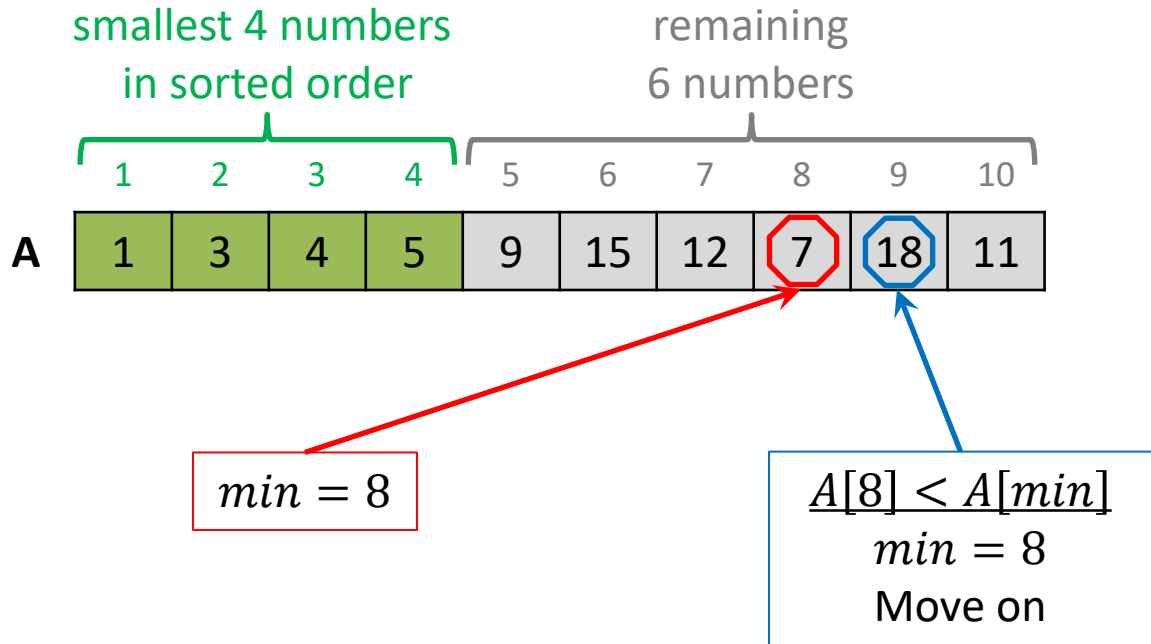
Suppose somehow we have this:



Selection Sort

State 4

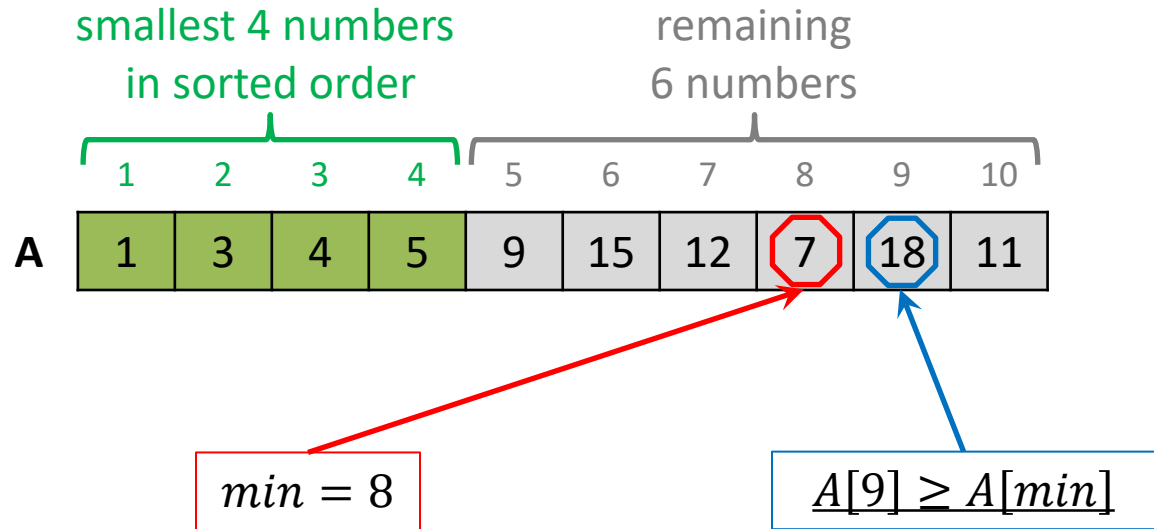
Suppose somehow we have this:



Selection Sort

State 4

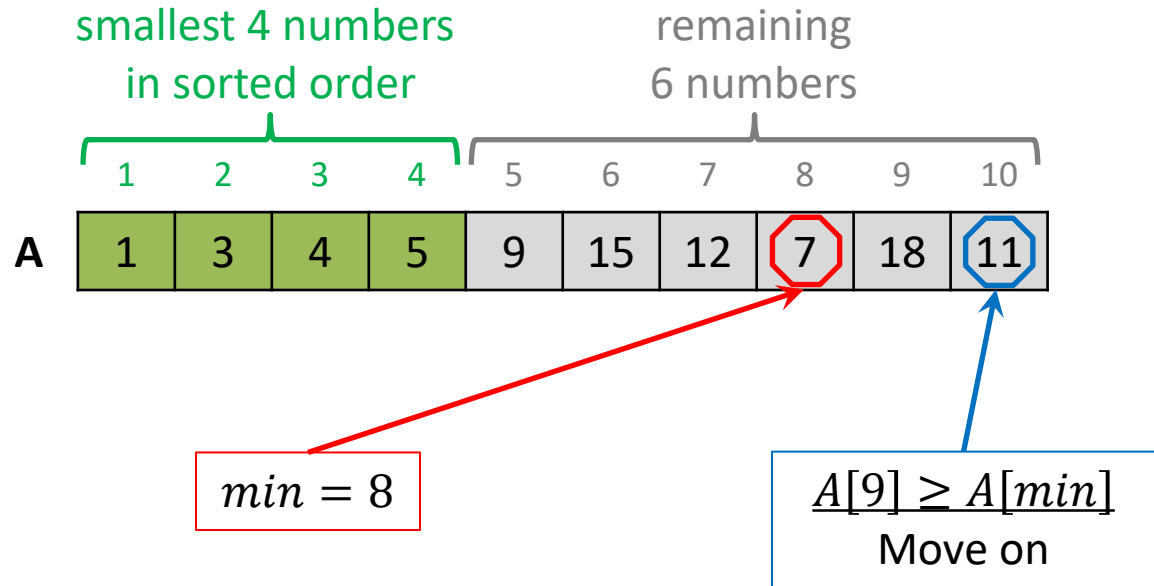
Suppose somehow we have this:



Selection Sort

State 4

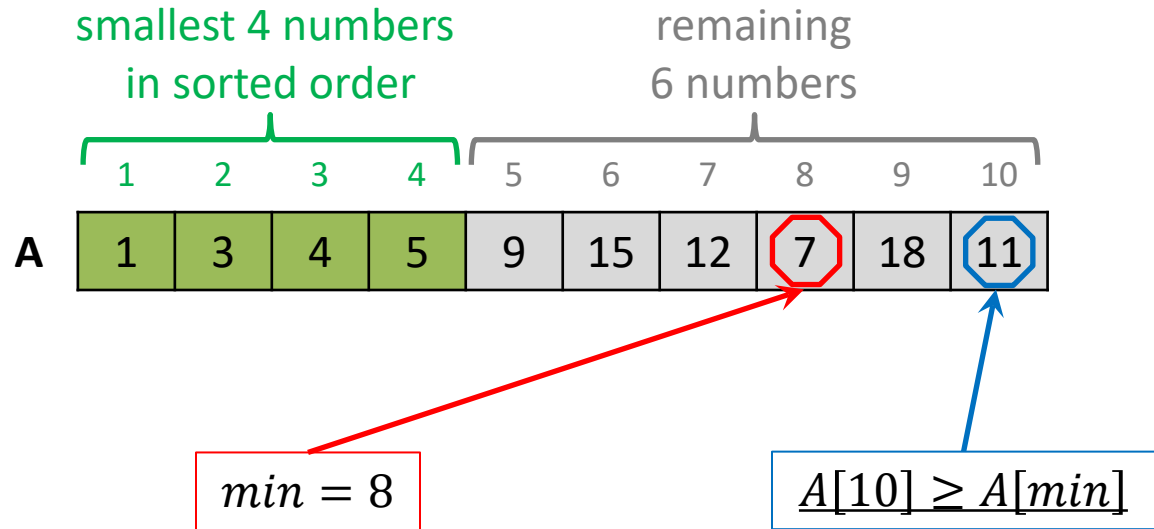
Suppose somehow
we have this:



Selection Sort

State 4

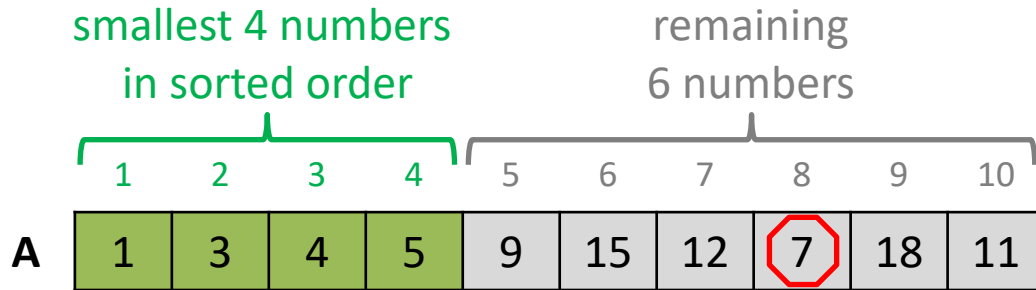
Suppose somehow we have this:



Selection Sort

State 4

Suppose somehow
we have this:



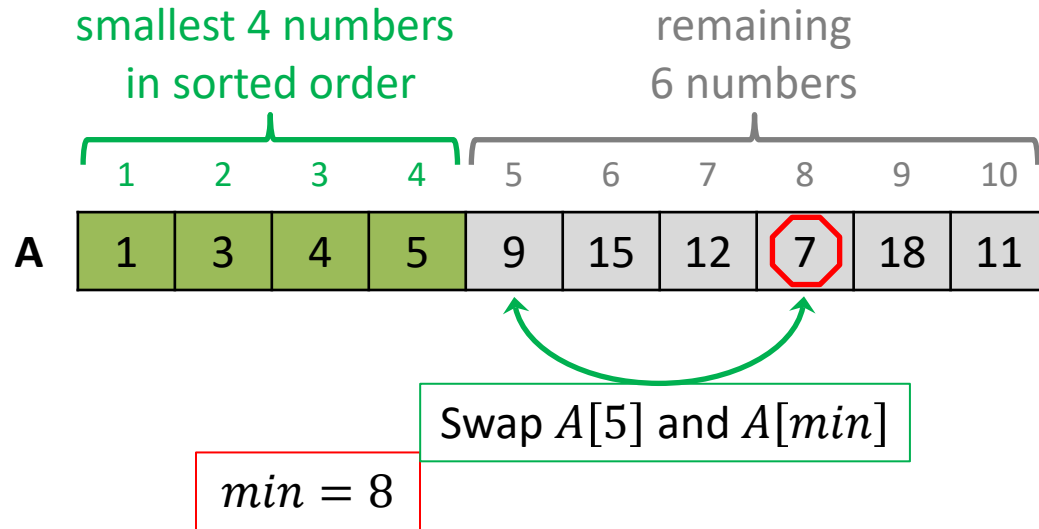
$min = 8$

$A[10] \geq A[min]$
Done scanning

Selection Sort

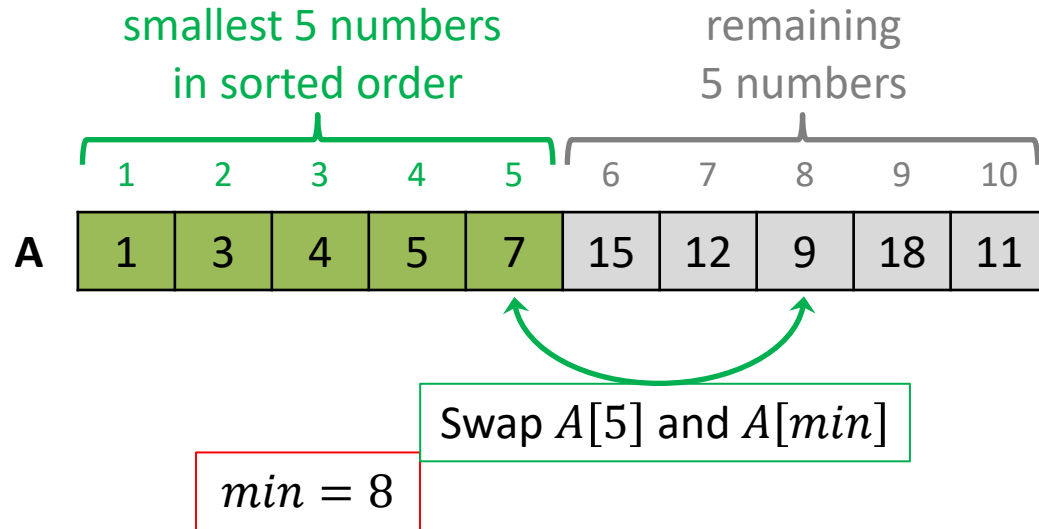
State 4

Suppose somehow we have this:



Selection Sort

State 5



Selection Sort

State 0

Input array

	1	2	3	4	5	6	7	8	9	10
A	7	4	1	15	9	5	12	3	18	11

↓ Swap $A[1]$ with the smallest number in $A[1..10]$

State 1

$A[1]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	4	7	15	9	5	12	3	18	11

↓ Swap $A[2]$ with the smallest number in $A[2..10]$

State 2

$A[1..2]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	3	7	15	9	5	12	4	18	11

↓ Swap $A[3]$ with the smallest number in $A[3..10]$

State 3

$A[1..3]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	3	4	15	9	5	12	7	18	11

↓ Swap $A[4]$ with the smallest number in $A[4..10]$

State 4

$A[1..4]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	3	4	5	9	15	12	7	18	11

↓ Swap $A[5]$ with the smallest number in $A[5..10]$

State 5

$A[1..5]$ is now sorted

	1	2	3	4	5	6	7	8	9	10
A	1	3	4	5	7	15	12	9	18	11

Selection Sort

State 5

$A[1..5]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	15	12	9	18	11



Swap $A[6]$ with the smallest number in $A[6..10]$

State 6

$A[1..6]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	12	15	18	11



Swap $A[7]$ with the smallest number in $A[7..10]$

State 7

$A[1..7]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	11	15	18	12



Swap $A[8]$ with the smallest number in $A[8..10]$

State 8

$A[1..8]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	11	12	18	15



Swap $A[9]$ with the smallest number in $A[9..10]$

State 9

$A[1..9]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	11	12	15	18



Do nothing!

State 10

$A[1..10]$ is now sorted

A

1	2	3	4	5	6	7	8	9	10
1	3	4	5	7	9	11	12	15	18

Selection Sort

Input: An array $A[1 : n]$ of n numbers.

Output: Elements of $A[1 : n]$ rearranged in non-decreasing order of value.

SELECTION-SORT (A)

1. **for** $j = 1$ **to** $A.length - 1$
2. // find the index of an entry with the smallest value in $A[j..A.length]$
3. $min = j$
4. **for** $i = j + 1$ **to** $A.length$
5. **if** $A[i] < A[min]$
6. $min = i$
7. // swap $A[j]$ and $A[min]$
8. $A[j] \leftrightarrow A[min]$

Proof of Correctness of Insertion Sort

Loop Invariants

We use *loop invariants* to prove correctness of iterative algorithms.

A loop invariant is associated with a given loop of an algorithm.

It is a formal statement about the relationship among variables of the algorithm such that

- [**Initialization**] It is true prior to the first iteration of the loop
- [**Maintenance**] If it is true before an iteration of the loop, it remains true before the next iteration
- [**Termination**] When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct

Loop Invariants for Insertion Sort

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$
2. $key = A[j]$
3. // insert $A[j]$ into the sorted sequence $A[1..j - 1]$
4. $i = j - 1$
5. **while** $i > 0$ **and** $A[i] > key$
6. $A[i + 1] = A[i]$
7. $i = i - 1$
8. $A[i + 1] = key$

Loop Invariants for Insertion Sort

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j - 1]$ consists of the elements
originally in $A[1..j - 1]$, but in sorted order

2. $key = A[j]$

3. $//$ insert $A[j]$ into the sorted sequence $A[1..j - 1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

6. $A[i + 1] = A[i]$

7. $i = i - 1$

8. $A[i + 1] = key$

Loop Invariants for Insertion Sort

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j - 1]$ consists of the elements originally in $A[1..j - 1]$, but in sorted order

2. $key = A[j]$

3. $//$ insert $A[j]$ into the sorted sequence $A[1..j - 1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i + 1] = A[i]$

7. $i = i - 1$

8. $A[i + 1] = key$

Loop Invariant 1: Initialization

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j - 1]$ consists of the elements
originally in $A[1..j - 1]$, but in sorted order

2. $key = A[j]$

3. // insert $A[j]$ into the sorted sequence $A[1..j - 1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i + 1] = A[i]$

7. $i = i - 1$

8. $A[i + 1] = key$

At the start of the first iteration of the loop (in lines 1 – 8): $j = 2$

Hence, subarray $A[1..j - 1]$ consists of a single element $A[1]$, which is in fact the original element in $A[1]$.

A subarray consisting of a single element is trivially sorted.

Hence, the invariant holds initially.

Loop Invariant 1: Maintenance

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j-1]$ consists of the elements
originally in $A[1..j-1]$, but in sorted order

2. $key = A[j]$

3. $//$ insert $A[j]$ into the sorted sequence $A[1..j-1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i+1] = A[i]$

7. $i = i - 1$

8. $A[i+1] = key$

We assume that invariant 1 holds before the start of the current iteration.

Hence, the following holds: $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$, but in sorted order.

For invariant 1 to hold before the start of the next iteration, the following must hold at the end of the current iteration:

$A[1..j]$ consists of the elements originally in $A[1..j]$, but in sorted order.

We use invariant 2 to prove this.

Loop Invariant 1: Maintenance

Loop Invariant 2: Initialization

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j-1]$ consists of the elements
originally in $A[1..j-1]$, but in sorted order

2. $key = A[j]$

3. $//$ insert $A[j]$ into the sorted sequence $A[1..j-1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i+1] = A[i]$

7. $i = i - 1$

8. $A[i+1] = key$

At the start of the first iteration of the loop (in lines 5 – 7): $i = j - 1$

Hence, subarray $A[i..j]$ consists of only two entries: $A[i]$ and $A[j]$.

We know the following:

- $A[i] > key$ (explicitly tested in line 5)
- $A[j] = key$ (from line 2)

Hence, invariant 2 holds initially.

Loop Invariant 1: Maintenance

Loop Invariant 2: Maintenance

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j-1]$ consists of the elements
originally in $A[1..j-1]$, but in sorted order

2. $key = A[j]$

3. $//$ insert $A[j]$ into the sorted sequence $A[1..j-1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i+1] = A[i]$

7. $i = i - 1$

8. $A[i+1] = key$

We assume that invariant 2 holds before the start of the current iteration.

Hence, the following holds: $A[i..j]$ are each $\geq key$.

Since line 6 copies $A[i]$ which is known to be $> key$ to $A[i+1]$ which also held a value $\geq key$, the following holds at the end of the current iteration: $A[i+1..j]$ are each $\geq key$.

Before the start of the next iteration the check $A[i] > key$ in line 5 ensures that invariant 2 continues to hold.

Loop Invariant 1: Maintenance

Loop Invariant 2: Maintenance

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j-1]$ consists of the elements
originally in $A[1..j-1]$, but in sorted order

2. $key = A[j]$

3. // insert $A[j]$ into the sorted sequence $A[1..j-1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i+1] = A[i]$

7. $i = i - 1$

8. $A[i+1] = key$

Observe that the inner loop (in lines 5 – 7) does not destroy any data because though the first iteration overwrites $A[j]$, that $A[j]$ has already been saved in key in line 2.

As long as key is copied back into a location in $A[1..j]$ without destroying any other element in that subarray, we maintain the invariant that $A[1..j]$ contains the first j elements of the original list.

Loop Invariant 1: Maintenance

Loop Invariant 2: Termination

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j-1]$ consists of the elements
originally in $A[1..j-1]$, but in sorted order

2. $key = A[j]$

3. // insert $A[j]$ into the sorted sequence $A[1..j-1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i+1] = A[i]$

7. $i = i - 1$

8. $A[i+1] = key$

When the inner loop terminates, we know the following.

- $A[1..i]$ is sorted with each element $\leq key$
 - if $i = 0$, true by default
 - if $i > 0$, true because $A[1..i]$ is sorted and $A[i] \leq key$
- $A[i+1..j]$ is sorted with each element $\geq key$ because the following held before i was decremented: $A[i..j]$ is sorted with each item $\geq key$
- $A[i+1] = A[i+2]$ if the loop was executed at least once, and $A[i+1] = key$ otherwise

Loop Invariant 1: Maintenance

Loop Invariant 2: Termination

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j-1]$ consists of the elements
originally in $A[1..j-1]$, but in sorted order

2. $key = A[j]$

3. $//$ insert $A[j]$ into the sorted sequence $A[1..j-1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i+1] = A[i]$

7. $i = i - 1$

8. $A[i+1] = key$

When the inner loop terminates, we know the following.

- $A[1..i]$ is sorted with each element $\leq key$
- $A[i+1..j]$ is sorted with each element $\geq key$
- $A[i+1] = A[i+2]$ or $A[i+1] = key$

Given the facts above, line 8 does not destroy any data, and gives us $A[1..j]$ as the sorted permutation of the original data in $A[1..j]$.

Loop Invariant 1: Termination

INSERTION-SORT (A)

1. **for** $j = 2$ **to** $A.length$

Invariant 1: $A[1..j-1]$ consists of the elements
originally in $A[1..j-1]$, but in sorted order

2. $key = A[j]$

3. // insert $A[j]$ into the sorted sequence $A[1..j-1]$

4. $i = j - 1$

5. **while** $i > 0$ **and** $A[i] > key$

Invariant 2: $A[i..j]$ are each $\geq key$

6. $A[i+1] = A[i]$

7. $i = i - 1$

8. $A[i+1] = key$

When the outer loop terminates, we know that $j = A.length + 1$.

Hence, $A[1..j-1]$ is the entire array $A[1..A.length]$, which is sorted and contains the original elements of $A[1..A.length]$.

*When the math
gets tough*

*Analysis of Algorithms
in
Rhymes and Pictures*

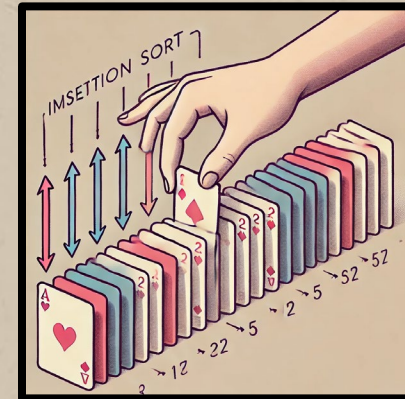
ChatGPT 4o mini

Insertion Sort: A Rhyming Tale

When sorting is what you need to do,
Insertion Sort will pull you through.
One by one, it takes each card,
And fits it in, though it seems hard.

The list is scanned from left to right,
And each new value finds its height.
Compared to those it left behind,
A better spot it soon will find.

At first, it's slow, but don't despair,
With fewer moves, it's light as air.
For smaller lists, it works just fine,
Its worst is n -squared every time.



Here is an illustration of the insertion sort process. You can see how the hand is placing a new card in the correct position among already sorted cards, with arrows showing comparisons and movements. This image brings the sorting algorithm to life visually!



Complexity Analysis in Rhyme

The **Best Case** is when things are set,
The list is sorted, smooth as a jet.
Just one pass, all values shine,
 $O(n)$ time, it's linear, fine!

The **Average Case** isn't so sweet,
Each value must find its seat.
On the whole, you'll find and see,
The complexity is $O(n^2)$, oh gee!

In the **Worst Case**, things go slow,
Reversing the list—what a woe!
 $O(n^2)$ again, a quadratic crime,
But still, it's sure, it sorts in time.

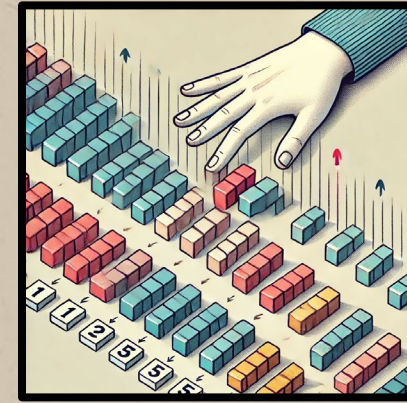


Selection Sort: A Rhyming Tale

When sorting's tough and you must choose,
Selection Sort will never lose.
It scans the list, it looks around,
For the smallest value to be found.

One by one, from start to end,
Each element must find a friend.
The smallest moves to the front row,
And then the rest continue to go.

It swaps, it shifts, but moves are few,
For larger lists, it might be slow too.
Yet steady it works, no trick or ploy,
Its simple logic you'll enjoy.



Here's an illustration of selection sort in action. The hand is selecting the smallest block from the unsorted section on the right, preparing to swap it to the front, while the blocks on the left are already sorted. The process is shown with arrows to indicate the scanning and swapping.



Complexity Analysis in Rhyme

The **Best Case**, Worst, or **Average Time**,
Selection Sort stays in its prime.
It always scans the list full through,
 $O(n^2)$ is what it will do.

For every element that's not set,
It picks the smallest it can get.
Comparisons? They pile and grow,
 $O(n^2)$ for all, that's just how it goes.

But swaps are few, not like before,
Just $O(n)$ swaps—no more, no more.
So though it may not be that fast,
Its ease and steadiness will last!

